

Python Code

```
1  import os
2  import duckdb
3  import logging
4
5  # Configure clean console logging
6  logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %
    (message)s')
7  log = logging.getLogger(__name__)
8
9  # --- Configuration & Paths ---
10 RAW_DIR = "./final_production_parquet"
11 CLEAN_DIR = "./lean_production_parquet"
12 TARGET_SAMPLE_SIZE = 1000000
13
14 def build_lean_dataset():
15     os.makedirs(CLEAN_DIR, exist_ok=True)
16     con = duckdb.connect()
17
18     log.info("Inspecting warehouse and calculating luxury outliers...")
19     price_cap = con.execute(f"SELECT quantile_cont(Price, 0.99) FROM
    read_parquet('{RAW_DIR}/*.parquet')").fetchone()[0]
20     log.info(f"99th Percentile Price Cap: £{price_cap:,.2f}")
21
22     # Inspect schema to find out exactly what columns are available
23     raw_columns = [col[0] for col in con.execute(f"DESCRIBE SELECT * FROM
    read_parquet('{RAW_DIR}/*.parquet') LIMIT 1").fetchall()]
24     log.info(f"DEBUG: Detected columns available in your Parquet files:
    {raw_columns}")
25
26     # Smart Date Column Resolver
27     date_col = None
28     possible_date_names = ["Date_of_Transfer", "date", "Date",
    "Transaction_Date", "Year", "year", "Transfer_Date"]
29     for name in possible_date_names:
30         if name in raw_columns:
31             date_col = name
32             break
33
34     # 1. Compile Global Target Encoding Maps based on LOG-PRICES to prevent
    leakages
35     log.info("Compiling global Target Encoding maps for Postcodes and
    Localities (Log-Scale)...")
36     con.execute(f"""
37         CREATE OR REPLACE TEMP TABLE postcode_map AS
38         SELECT Postcode,
39             AVG(LN(Price)) AS Postcode_Mean_Log_Price, -- Base features on
    Log scale
40             COUNT(Price) AS Postcode_Volume
41         FROM read_parquet('{RAW_DIR}/*.parquet')
42         WHERE Price <= {price_cap} AND Postcode IS NOT NULL
```

```

43         GROUP BY Postcode;
44     """
45
46     con.execute(f"""
47         CREATE OR REPLACE TEMP TABLE locality_map AS
48         SELECT Locality, AVG(LN(Price)) AS Locality_Mean_Log_Price  -- Base
features on Log scale
49         FROM read_parquet('{RAW_DIR}/*.parquet')
50         WHERE Price <= {price_cap} AND Locality IS NOT NULL
51         GROUP BY Locality;
52     """)
53
54     # 2. Extract sample and conditionally build temporal features
55     log.info(f"Streaming {TARGET_SAMPLE_SIZE:,} log-normalized hyper-
engineered rows directly to disk...")
56
57     if date_col:
58         log.info(f"Success: Mapping temporal cyclical vectors using column
'{date_col}'")
59         time_select_logic = f"""
60             EXTRACT(YEAR FROM CAST(main.{date_col} AS DATE)) AS
Transaction_Year,
61             SIN(2 * PI() * EXTRACT(MONTH FROM CAST(main.{date_col} AS DATE)) /
12) AS Month_Sin,
62             COS(2 * PI() * EXTRACT(MONTH FROM CAST(main.{date_col} AS DATE)) /
12) AS Month_Cos,
63             """
64         time_where_logic = f"AND main.{date_col} IS NOT NULL"
65     else:
66         log.warning("CRITICAL: No date tracking column discovered in your
Parquet schema. Defaulting to baseline spatial tracking features.")
67         time_select_logic = """
68             2026 AS Transaction_Year,
69             0.0 AS Month_Sin,
70             1.0 AS Month_Cos,
71             """
72         time_where_logic = ""
73
74     # Value fallback using Log of half the price cap
75     log_fallback_value = con.execute(f"SELECT LN({price_cap} *
0.5)").fetchone()[0]
76
77     query = f"""
78     COPY (
79         SELECT
80             LN(main.Price) AS Log_Price,  -- TARGET MUTATION: Changes raw
target from Price to Log_Price
81             main.Duration,
82             main.PPD_Category,
83
84             -- Location Baselines & Volume Density matching Log Space
85             COALESCE(p.Postcode_Mean_Log_Price, {log_fallback_value}) AS
Postcode_Value,

```

```

86         COALESCE(l.Locality_Mean_Log_Price, {log_fallback_value}) AS
Locality_Value,
87         COALESCE(p.Postcode_Volume, 1) AS Postcode_Density,
88
89         -- Spatial Interaction Ratio (Calculated safely using log
differences)
90         CASE
91             WHEN l.Locality_Mean_Log_Price IS NOT NULL AND
l.Locality_Mean_Log_Price > 0
92             THEN COALESCE(p.Postcode_Mean_Log_Price,
l.Locality_Mean_Log_Price) / l.Locality_Mean_Log_Price
93             ELSE 1.0
94         END AS Local_Premium_Ratio,
95
96         -- Inject dynamic time logic block
97         {time_select_logic}
98
99         -- Raw keys for Azure AutoML to identify regional variations
100        main.District,
101        main.Street
102
103        FROM read_parquet('{RAW_DIR}/*.parquet') main
104        LEFT JOIN postcode_map p ON main.Postcode = p.Postcode
105        LEFT JOIN locality_map l ON main.Locality = l.Locality
106        WHERE main.Price <= {price_cap}
107              AND main.Postcode IS NOT NULL
108              {time_where_logic}
109        USING SAMPLE {TARGET_SAMPLE_SIZE} rows
110    ) TO '{CLEAN_DIR}/lean_housing_data.parquet' (FORMAT 'PARQUET',
COMPRESSION 'ZSTD');
111    """
112
113    con.execute(query)
114    log.info(f"Successfully generated clean target-encoded dataset at
{CLEAN_DIR}")
115
116    if __name__ == "__main__":
117        build_lean_dataset()

```